

Unit-2

Practical No.1

Definition : Write a program to demonstrate conditional statements using if, if-else, and if-elif-else.

Aim/ Objective:

- To understand the use of conditional statements in Python.
- To use the if statement for decision-making.
- To use the if-else statement to execute one of two blocks.
- To use the if-elif-else statement for multiple conditions.
- To understand how conditions control the flow of a Python program.

Source Code:

```
num = int(input("enter the number :"))
```

```
if num > 0:  
    print("number is postive")  
elif num <0 :  
    print("number is negative")  
else:  
    print("numebr is Zero")
```

Output:

```
enter the number :56  
number is postive
```

Conclusion:

The program successfully demonstrates the use of if, if-else, and if-elif-else conditional statements in Python. It shows how conditions are used to make decisions and control the flow of a program.

Practical No.2

Definition : Write a program to check whether a number is positive, negative, or zero using nested conditions.

Aim/ Objective:

- To understand the concept of nested conditional statements in Python.
- To accept a number from the user.
- To check whether the number is positive, negative, or zero.
- To understand how one condition can be placed inside another condition.

Source Code:

```
num =int(input("enter the number"))

if num>=0:
    if num ==0:
        print("the number is zero")
    else:
        print("the number is Positive")
else:
    print("the number is Negative")
```

Output:

```
enter the number0
the number is zero
```

Conclusion:

The program successfully checks whether a given number is positive, negative, or zero using nested conditions. It demonstrates how nested if statements can be used to make multiple decisions in Python.

Practical No.3

Definition : Write a program to generate a multiplication table using a for loop.

Aim/ Objective:

- To understand the use of for loop in Python.
- To accept a number from the user.
- To generate the multiplication table of the given number.
- To understand iteration using a for loop.

Source Code:

```
num = int(input("enter the number"))

for i in range(1,11):
    print(num,"x",i,"=",num * i)
```

Output:

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
```

$$5 \times 9 = 45$$

$$5 \times 10 = 50$$

Conclusion:

The program successfully generates the multiplication table of a given number using a for loop. It demonstrates how a for loop is used to repeat a set of statements multiple times in Python.

Practical No.4

Definition : Write a program to find the sum of digits of a number using a while loop.

Aim/ Objective:

- To understand the use of while loop in Python.
- To accept a number from the user.
- To extract individual digits from a number.
- To calculate the sum of all digits using a while loop.

Source Code:

```
num = int(input("enter the number"))  
num=abs(num)
```

```
sum_digits = 0
```

```
while num>0:  
    digit = num%10  
    sum_digits += digit  
    num //= 10
```

```
print("sum of digits",sum_digits)
```

Output:

```
enter the number66  
sum of digits 12
```

Conclusion:

The program successfully finds the sum of digits of a given number using a while loop. It demonstrates how digits can be extracted and added one by one using arithmetic operations in Python.

Practical No.5

Definition : Write a program to demonstrate the use of break, continue, and pass statements in Python.

Aim/ Objective:

- To understand the use of break statement in Python.
- To understand the use of continue statement.
- To understand the use of pass statement.
- To demonstrate how these statements control the flow of a loop.

Source Code:

```
for i in range(1,6):  
    if i==3:  
        break  
    print(i)
```

```
for i in range(1,6):  
    if i ==3:  
        continue  
    print(i)
```

```
for i in range(1,6):  
    if i==3:  
        pass  
    print(i)
```

Output:

```
1  
2  
1  
2  
4  
5  
1  
2  
3  
4  
5
```

Conclusion:

The program successfully demonstrates the use of break, continue, and pass statements in Python. The break statement stops the loop, continue skips the current iteration, and pass

does nothing and allows the program to continue normally.

Practical No.6

Definition : Write a program to iterate over lists, strings, and dictionaries using loops.

Aim/ Objective:

- To understand how to iterate over a list using a loop.
- To understand how to iterate over characters of a string.
- To understand how to iterate over a dictionary using loops.
- To understand the use of for loop with different Python data types.

Source Code:

```
num =[1,2,3]
```

```
for i in num:  
    print(i)
```

```
name = "python"
```

```
for i in name:  
    print(i)
```

```
student = {"name": "Harsh", "age": 21}
```

```
for key, value in student.items():  
    print(key, value)
```

Output:

```
1  
2  
3  
p  
y  
t  
h  
o  
n  
name Vishw  
age 21
```

Conclusion:

The program successfully demonstrates how to iterate over lists, strings, and dictionaries using for loops in Python. It shows how loops can be used to access each element or

key-value pair one by one.

Practical No.7

Definition : Write a program to demonstrate list, dictionary, and set comprehensions in Python.

Aim/ Objective:

- To understand the concept of comprehensions in Python.
- To create a list using list comprehension.
- To create a dictionary using dictionary comprehension.
- To create a set using set comprehension.
- To write concise and readable Python code.

Source Code:

```
num = [1,2,3,4,5,56,65,43]
```

```
squ = [x * x for x in num]
```

```
print("squ",squ)
```

```
squ_dict = {x: x * x for x in num}
```

```
print("squ_dict",squ_dict)
```

```
squ_set = {x * x for x in num}
```

```
print("squ_set",squ_set)
```

Output:

```
squ [1, 4, 9, 16, 25, 3136, 4225, 1849]
```

```
squ_dict {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 56: 3136, 65: 4225, 43: 1849}
```

```
squ_set {3136, 1, 4225, 4, 9, 16, 1849, 25}
```

Conclusion:

The program successfully demonstrates list, dictionary, and set comprehensions in Python. Comprehensions provide a simple and concise way to create collections using loops and expressions.

Practical No.8

Definition : Write a program to illustrate variable scope using local, global, and nonlocal variables.

Aim/ Objective:

- To understand the concept of variable scope in Python.
- To understand the use of local variables.
- To understand the use of global variables.
- To understand the use of nonlocal variables.
- To understand how variables are accessed inside different functions.

Source Code:

```
x = 10

def outer():
    y = 20

    def inner():
        nonlocal y
        y = 30
        print("Nonlocal:", y)

    inner()

    def local():
        z = 40
        print("Local:", z)

    local()

print("Global:", x)
outer()
```

Output:

```
Global: 10
Nonlocal: 30
Local: 40
```

Conclusion:

The program successfully demonstrates variable scope in Python. The global variable can be accessed throughout the program, the nonlocal variable is accessed from an outer function, and the local variable is available only inside the function where it is defined.

Practical No.9

Definition : Write a program to demonstrate iterators and iterables in Python.

Aim/ Objective:

- To understand the concept of iterables in Python.
- To understand the concept of iterators.
- To use the iter() function to create an iterator.
- To use the next() function to access elements one by one.
- To understand the difference between an iterable and an iterator.

Source Code:

```
numbers = [10, 20, 30]
```

```
for i in numbers:  
    print(i)
```

```
it = iter(numbers)
```

```
print(next(it))  
print(next(it))  
print(next(it))
```

Output:

```
10  
20  
30  
10  
20  
30
```

Conclusion:

The program successfully demonstrates iterables and iterators in Python. A list is an iterable, and the iter() function converts it into an iterator. The next() function is then used to access each element one by one.

Practical No.10

Definition : Write a program to generate a sequence of numbers using generator functions and the yield keyword.

Aim/ Objective:

- To understand the concept of generator functions in Python.
- To understand the use of the yield keyword.
- To generate a sequence of numbers one at a time.
- To understand how generators save memory by producing values when needed.

Source Code:

```
def numbers():  
    yield 1  
    yield 2  
    yield 3  
    yield 4  
    yield 5
```

```
for i in numbers():  
    print(i)
```

Output:

```
1  
2  
3  
4  
5
```

Conclusion:

The program successfully demonstrates the use of a generator function and the yield keyword in Python. The generator produces numbers one at a time, making it memory efficient and useful for generating large sequences.